

BAB 6

STATE DAN INTERAKSI PENGGUNA

A. Tujuan Pembelajaran

Memahami konsep state dalam Flutter dan bagaimana cara mengelola keadaan dalam aplikasi menggunakan stateful widget.

B. Memahami Konsep State Di Flutter

Di dalam Flutter, terdapat dua jenis state utama yang perlu dipahami: stateless dan stateful. Stateless widget adalah widget yang tidak memiliki data atau keadaan internal yang berubah sepanjang waktu. Mereka hanya bergantung pada input yang diberikan dan menampilkan tampilan yang sesuai berdasarkan input tersebut. Sebaliknya, stateful widget adalah widget yang memiliki data atau keadaan internal yang dapat berubah. Mereka dapat merespons perubahan input atau interaksi pengguna dengan memperbarui tampilan berdasarkan keadaan yang berubah.

Penggunaan stateful widget memerlukan pemahaman tentang lifecycle widget. Setiap stateful widget memiliki metode khusus yang dipanggil dalam siklus hidupnya, seperti **initState()** yang dipanggil sekali ketika widget pertama kali dibuat, dan **setState()** yang digunakan untuk memperbarui tampilan widget ketika keadaan internal berubah. Memahami lifecycle ini penting untuk menjaga konsistensi antara data dan tampilan dalam aplikasi Flutter.

Flutter menggunakan pendekatan “pemrograman reaktif ” untuk mengelola state, di mana UI merespons perubahan state secara otomatis. Ini berarti ketika state berubah, Flutter secara otomatis membangun ulang bagian-bagian UI yang berdampak dan memperbarui tampilannya. Pendekatan ini membuat pengembangan UI menjadi lebih mudah karena Anda tidak perlu secara manual memperbarui UI setiap kali state berubah.

Salah satu pendekatan yang umum digunakan untuk mengelola state di Flutter adalah dengan menggunakan provider. Provider adalah paket Flutter yang digunakan untuk mengelola state dan berbagi data antara widget di seluruh aplikasi. Dengan menggunakan provider, Anda dapat mengatur state aplikasi secara terpusat dan memperbarui tampilan secara otomatis ketika state berubah.

Selain itu, Flutter juga menyediakan widget ValueNotifier dan ChangeNotifier yang dapat digunakan untuk mengelola state lokal di dalam widget. ValueNotifier digunakan untuk menyimpan dan memperbarui nilai tunggal, sedangkan ChangeNotifier digunakan untuk menyimpan state yang kompleks dan memberikan notifikasi ketika state berubah.

Memahami konsep state di Flutter juga melibatkan pengelompokan state menjadi dua jenis: state lokal dan state global. State lokal adalah state yang terkait dengan satu widget atau subtree widget tertentu, sedangkan state global adalah state yang dapat diakses dan diubah dari mana saja dalam aplikasi. Penggunaan state lokal dan global harus dipertimbangkan sesuai dengan kebutuhan aplikasi dan kompleksitasnya.

Penting untuk diingat bahwa penggunaan state dalam Flutter harus dikelola dengan hati-hati untuk menghindari masalah seperti state yang tidak konsisten atau overhead yang berlebihan. Anda harus mempertimbangkan berbagai faktor seperti kompleksitas aplikasi, kinerja, dan pengalaman pengguna saat memilih pendekatan pengelolaan state yang tepat.

Ketika mengembangkan aplikasi Flutter, pemahaman yang baik tentang konsep state adalah kunci untuk membuat aplikasi yang responsif, dinamis, dan mudah dipelihara. Dengan menggunakan pendekatan yang tepat untuk mengelola state dan memperbarui tampilan sesuai kebutuhan, Anda dapat menciptakan pengalaman pengguna yang luar biasa dan memuaskan.

C. Memperbarui Tampilan Dengan `setState()`

Memperbarui tampilan dengan `setState()` adalah konsep kunci dalam pengembangan aplikasi Flutter yang dinamis. Dalam Flutter, `setState()` adalah metode yang digunakan untuk memberitahu framework bahwa state internal dari suatu widget telah berubah dan perlu membangun ulang tampilan widget tersebut. Ketika Anda memanggil `setState()`, Flutter akan menjadwalkan ulang pemanggilan `build()` pada widget yang bersangkutan, sehingga memperbarui tampilannya sesuai dengan perubahan state.

Penggunaan `setState()` sangat berguna ketika Anda memiliki stateful widget yang bergantung pada keadaan internal yang dapat berubah, misalnya ketika ada perubahan data yang diterima dari sumber eksternal seperti interaksi pengguna atau pembaruan data dari server. Dengan memanggil `setState()` setelah perubahan state, Anda memastikan bahwa tampilan widget diperbarui sesuai dengan keadaan yang baru.

Penting untuk dipahami bahwa setiap pemanggilan `setState()` menyebabkan seluruh widget subtree yang bersangkutan dibangun ulang. Ini termasuk widget-widjet turunan dari widget yang memanggil `setState()`. Oleh karena itu, Anda harus memperhatikan kinerja dan efisiensi saat menggunakan `setState()` dalam aplikasi Flutter Anda.

Pemanggilan `setState()` harus dilakukan secara asinkron, yang berarti bahwa pemanggilan `build()` selanjutnya akan dijadwalkan untuk dijalankan di urutan berikutnya setelah metode saat ini selesai dieksekusi. Ini memastikan bahwa pembaruan tampilan tidak menghalangi eksekusi kode lain dalam aplikasi Anda.

Selain pembaruan tampilan, `setState()` juga dapat digunakan untuk memicu tindakan tambahan setelah pembaruan tampilan selesai, seperti menampilkan pesan pemberitahuan kepada pengguna atau memicu pembaruan data lainnya.

Penggunaan yang cerdas dari `setState()` sangat penting untuk menghindari membangun ulang tampilan yang tidak perlu atau

mempengaruhi kinerja aplikasi secara negatif. Anda harus memanggil **setState()** hanya ketika ada perubahan yang benar-benar mempengaruhi tampilan widget atau ketika Anda memerlukan pembaruan UI secara dinamis.

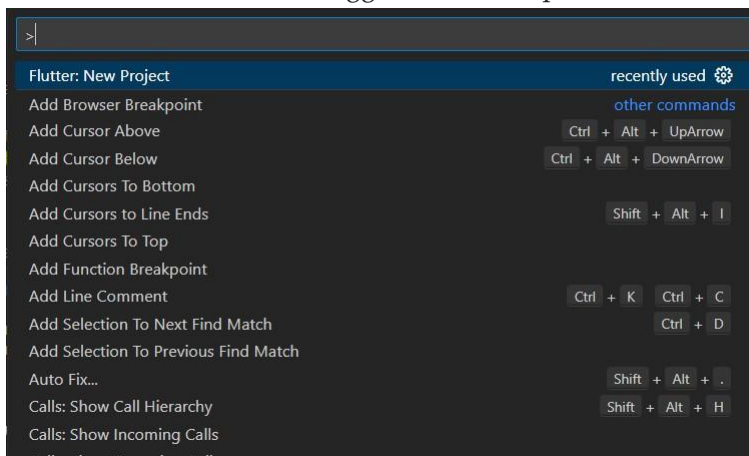
Memahami konsep **setState()** adalah kunci dalam mengembangkan aplikasi Flutter yang responsif dan dinamis. Dengan menggunakan metode ini dengan bijaksana, Anda dapat menciptakan pengalaman pengguna yang mulus dan memuaskan dalam aplikasi Anda.

Selain itu, penting juga untuk memahami bahwa **setState()** memungkinkan Anda untuk mengatur tampilan dengan responsif sesuai dengan perubahan state, sehingga memungkinkan Anda untuk membuat aplikasi Flutter yang interaktif dan dinamis.

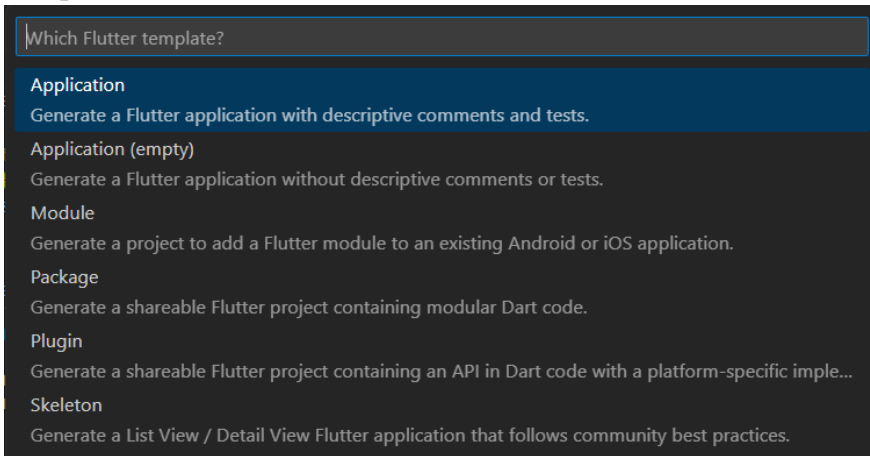
Ketika Anda menggunakan **setState()**, pastikan untuk memperbarui state dengan benar dan memastikan bahwa setiap pemanggilan **setState()** sesuai dengan perubahan yang terjadi dalam aplikasi Anda. Dengan demikian, Anda dapat memanfaatkan kekuatan Flutter untuk membuat aplikasi yang mengesankan bagi pengguna Anda.

Contoh coding:

Buat project baru menggunakan **Command Palette**. Pilih menu **View** dan klik **Command Palette** sehingga muncul tampilan berikut.

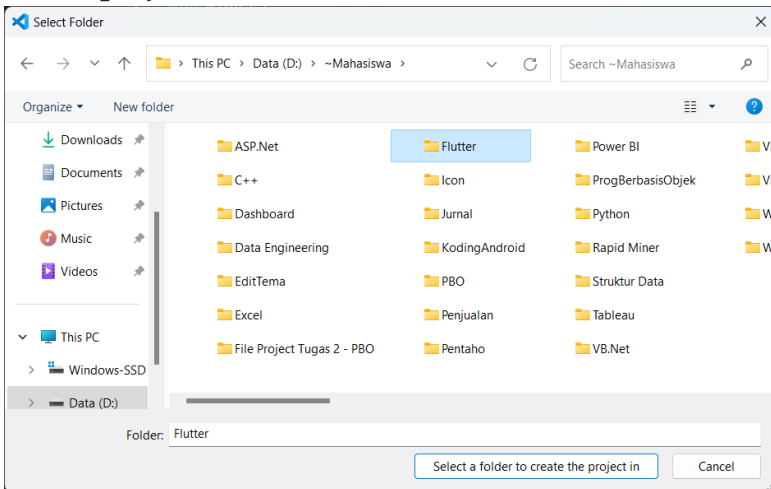


Kemudian ketikkan flutter, pilih **Flutter New Project**, maka akan muncul tampilan berikut.



Pilih menu **Application**.

Pilih folder yang diinginkan, kemudian klik tombol **Select a folder to create the project in**.



Beri nama projectnya, dengan mengetik **flutter_state** kemudian tekan **Enter**.

Buka file **lib/main.dart**, hapus semua code yang ada karena kita akan memulainya dari awal. Kemudian masukkan code berikut:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'SetState Example',
      home: CounterPage(),
    );
  }
}

class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  // ignore: library_private_types_in_public_api
  _CounterPageState createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }
}
```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Counter Example'),
    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          const Text(
            'Counter:',
          ),
          Text(
            '$_counter',
            style: Theme.of(context).textTheme.headlineMedium,
          ),
        ],
      ),
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: _incrementCounter,
      tooltip: 'Increment',
      child: const Icon(Icons.add),
    ),
  );
}

```

Dalam contoh di atas, kita memiliki sebuah stateful widget bernama **CounterPage** yang memiliki state `_counter` yang bertipe integer. Ketika tombol floating action button ditekan, metode **`_incrementCounter()`** dipanggil yang memanggil **`setState()`** untuk memperbarui nilai `_counter` dan memicu pembaruan tampilan widget.

Ketika **`setState()`** dipanggil, Flutter menjadwalkan ulang pemanggilan **`build()`** pada widget **`_CounterPageState`**, sehingga tampilan widget diperbarui sesuai dengan nilai **`_counter`** yang baru. Sebagai hasilnya,

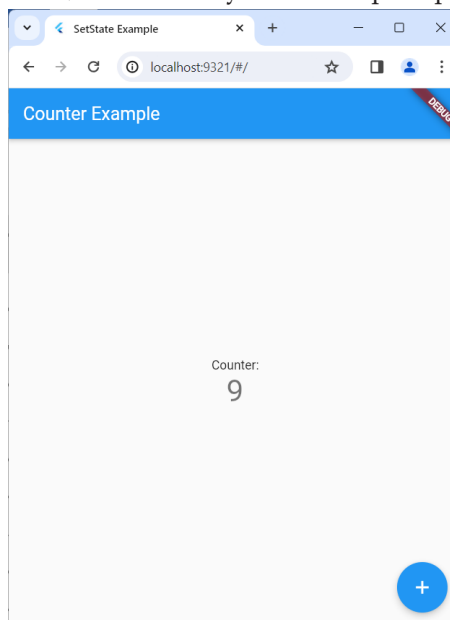
setiap kali tombol ditekan, nilai **_counter** bertambah dan tampilan widget diperbarui untuk mencerminkan perubahan tersebut.

Contoh ini mengilustrasikan bagaimana **setState()** digunakan untuk memperbarui tampilan widget secara dinamis dalam respons terhadap perubahan state, sehingga memungkinkan Anda untuk menciptakan antarmuka pengguna yang interaktif dan responsif dalam aplikasi Flutter Anda.

Untuk menjalankan koding, menggunakan terminal. Klik **New Terminal**. Kemudian ketikkan **flutter run**, tekan **Enter**. Pilih salah emulator yang akan digunakan untuk menjalankan project. Disini penulis menggunakan Chrome, berarti ketik angka 2.

```
PS D:\Mahasiswa\Flutter\aplikasipertama> flutter run
Multiple devices found:
Windows (desktop) • windows • windows-x64 • Microsoft Windows [Version 10.0.22621.1413]
Chrome (web)      • chrome • web-javascript • Google Chrome 111.0.5563.147
Edge (web)        • edge • web-javascript • Microsoft Edge 111.0.1661.62
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (To quit, press "q/Q"): 2
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...
```

Jika tidak ada masalah, maka hasilnya akan tampil seperti berikut.



D. Memproses Masukan Pengguna

Memproses masukan pengguna adalah salah satu aspek penting dalam pengembangan aplikasi Flutter yang interaktif dan responsif. Di dalam Flutter, ada berbagai cara untuk memproses masukan pengguna, seperti input teks, tap, swipe, dan banyak lagi, dan setiap jenis masukan ini dapat memicu tindakan yang berbeda dalam aplikasi Anda.

Salah satu cara yang umum digunakan untuk memproses masukan pengguna adalah melalui widget interaktif seperti tombol, field input, atau widget gesture seperti **GestureDetector**. Tombol digunakan untuk menanggapi ketika pengguna melakukan tap pada mereka, sementara field input memungkinkan pengguna memasukkan data seperti teks. Widget gesture seperti **GestureDetector** memungkinkan Anda menanggapi berbagai gerakan pengguna, seperti tap, swipe, atau drag.

Penggunaan widget seperti **InkWell**, **RaisedButton**, atau **GestureDetector** memungkinkan Anda menambahkan perilaku interaktif ke dalam aplikasi Anda dengan mudah. Anda dapat menetapkan fungsi atau metode ke properti seperti `onPressed` atau `onTap` pada widget-widget ini untuk menentukan tindakan yang akan dilakukan ketika pengguna melakukan interaksi.

Selain itu, Anda juga dapat menggunakan widget seperti **TextField** untuk memungkinkan pengguna memasukkan teks ke dalam aplikasi Anda. Anda dapat mengakses nilai yang dimasukkan oleh pengguna melalui properti seperti `onChanged` atau `onSubmit`, dan kemudian memprosesnya sesuai kebutuhan aplikasi Anda.

Flutter juga menyediakan widget khusus untuk memproses masukan pengguna yang lebih kompleks, seperti **Checkbox**, **RadioButton**, atau **Slider**. Widget-widget ini memungkinkan Anda untuk membuat antarmuka pengguna yang lebih interaktif dan dinamis, dan Anda dapat menanggapi perubahan nilai yang dihasilkan oleh pengguna dengan cara yang sesuai.

Penting untuk mempertimbangkan responsif aplikasi Anda terhadap masukan pengguna dengan benar. Ini mencakup menanggapi masukan pengguna dengan cepat dan memberikan umpan balik yang jelas kepada pengguna ketika mereka berinteraksi dengan aplikasi Anda.

Ketika memproses masukan pengguna, penting untuk memvalidasi dan membersihkan data yang dimasukkan oleh pengguna sebelum menggunakannya dalam aplikasi Anda. Ini membantu mencegah kesalahan atau masalah keamanan yang mungkin timbul akibat data yang tidak valid atau berbahaya.

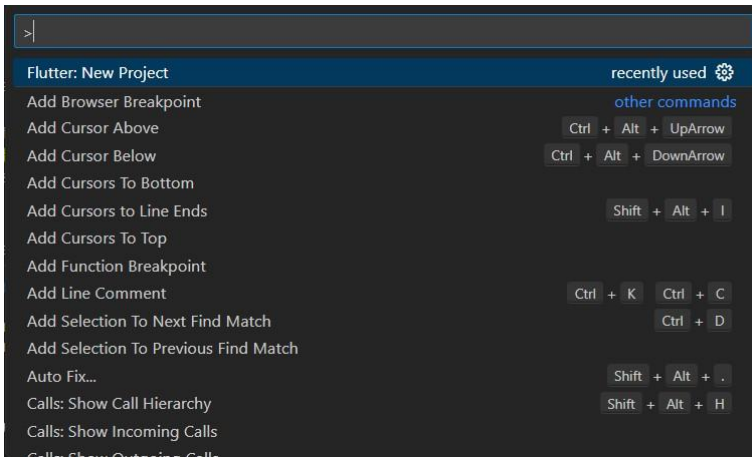
Selain itu, Anda juga dapat menggunakan animasi dan efek visual untuk meningkatkan pengalaman pengguna saat memproses masukan mereka. Misalnya, Anda dapat menggunakan animasi untuk memberikan umpan balik visual ketika pengguna menekan tombol atau memasukkan data ke dalam aplikasi Anda.

Penggunaan instrumen pengembangan seperti Widget Inspector atau Flutter DevTools dapat membantu Anda memahami dan menguji interaksi pengguna dalam aplikasi Anda. Ini memungkinkan Anda untuk melihat hierarki widget, menganalisis kinerja, dan memecahkan masalah yang mungkin timbul dalam respons terhadap masukan pengguna.

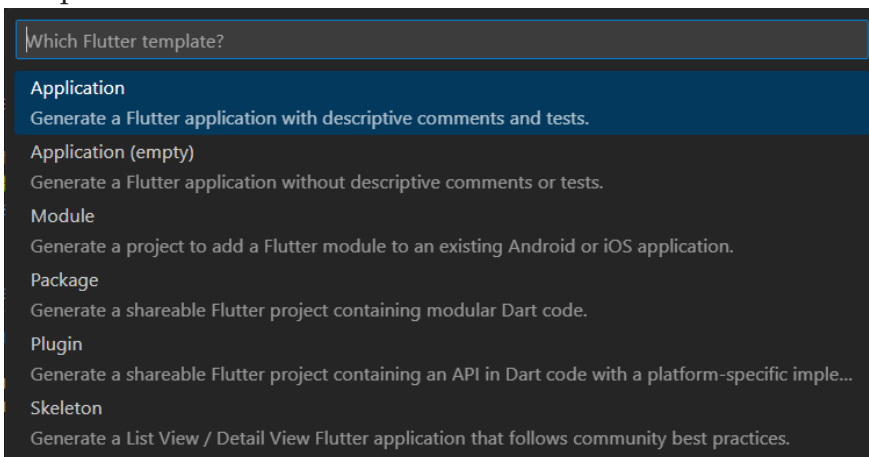
Memproses masukan pengguna dengan baik adalah kunci untuk menciptakan pengalaman pengguna yang menyenangkan dan produktif dalam aplikasi Flutter Anda. Dengan menggunakan berbagai widget dan teknik yang tersedia, Anda dapat membuat antarmuka pengguna yang interaktif dan responsif yang memenuhi kebutuhan pengguna Anda.

Contoh koding:

Buat project baru menggunakan **Command Palette**. Pilih menu **View** dan klik **Command Palette** sehingga muncul tampilan berikut.



Kemudian ketikkan flutter, pilih **Flutter New Project**, maka akan muncul tampilan berikut.



Pilih menu **Application**.

Pilih folder yang diinginkan, kemudian klik tombol **Select a folder to create the project in**.

```

class UserInputPage extends StatefulWidget {
  const UserInputPage({super.key});

  @override
  // ignore: library_private_types_in_public_api
  _UserInputPageState createState() => _UserInputPageState();
}

class _UserInputPageState extends State<UserInputPage> {
  String _inputText = "";

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('User Input Example'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(20.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            TextField(
              onChanged: (value) {
                setState(() {
                  _inputText = value;
                });
              },
              decoration: const InputDecoration(
                labelText: 'Enter some text',
                border: OutlineInputBorder(),
              ),
            ),
          ],
        ),
      ),
    );
  }
}

```

```

decoration: const InputDecoration(
  labelText: 'Enter some text',
  border: OutlineInputBorder(),
),
),

```

Dalam contoh di atas, kita menggunakan widget `TextField` untuk memungkinkan pengguna memasukkan teks. Saat pengguna memasukkan teks, kita menggunakan properti **`onChanged`** untuk menetapkan fungsi yang akan dipanggil setiap kali teks berubah. Di dalam fungsi tersebut, kita memanggil **`setState()`** untuk memperbarui tampilan dan menetapkan nilai `_inputText` sesuai dengan teks yang dimasukkan oleh pengguna.

Kemudian, kita menampilkan teks yang dimasukkan oleh pengguna di bawah `TextField` menggunakan widget `Text`. Nilai `_inputText` diperbarui setiap kali pengguna memasukkan atau mengubah teks, sehingga tampilan `Text` juga diperbarui secara otomatis untuk mencerminkan perubahan tersebut.

Dengan menggunakan widget `TextField` dan properti **`onChanged`**, kita dapat memproses masukan pengguna dalam aplikasi kita. Anda dapat menyesuaikan logika di dalam **`onChanged`** sesuai dengan kebutuhan aplikasi Anda, misalnya untuk validasi data atau pemrosesan tambahan sebelum menggunakan data yang dimasukkan oleh pengguna.

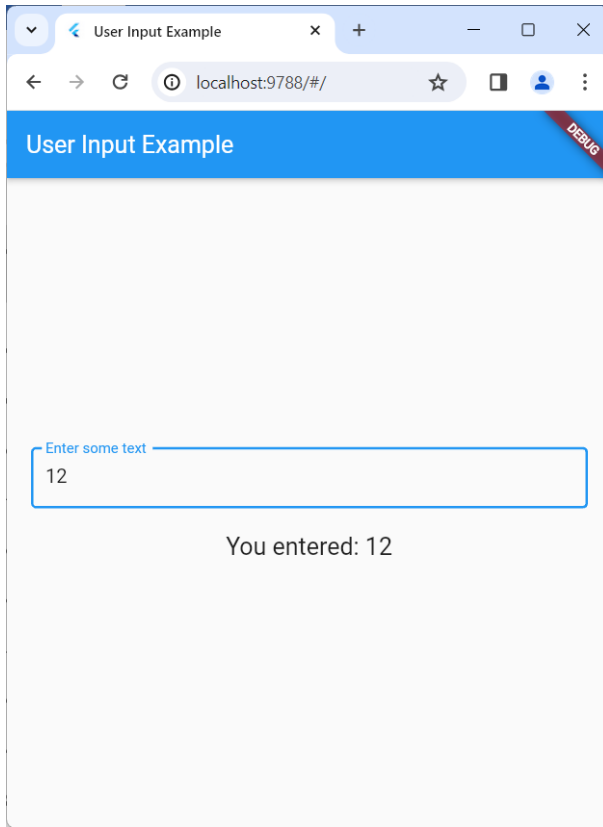
Untuk menjalankan koding, menggunakan terminal. Klik **New Terminal**. Kemudian ketikkan **`flutter run`**, tekan **Enter**. Pilih salah emulator yang akan digunakan untuk menjalankan project. Disini penulis menggunakan Chrome, berarti ketik angka 2.

```

PS D:\Mahasiswa\Flutter\aplikasipertama> flutter run
Multiple devices found:
Windows (desktop) • windows • windows-x64 • Microsoft Windows [Version 10.0.22621.1413]
Chrome (web)      • chrome • web-javascript • Google Chrome 111.0.5563.147
Edge (web)        • edge   • web-javascript • Microsoft Edge 111.0.1661.62
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (To quit, press "q/Q"): 2
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...

```

Jika tidak ada masalah, maka hasilnya akan tampil seperti berikut.



E. Menggunakan Fungsi Callback

Sebuah fungsi callback adalah fungsi yang diteruskan sebagai argumen ke fungsi lain, dan akan dipanggil oleh fungsi lain tersebut pada waktu tertentu. Dengan kata lain, Anda memberikan suatu fungsi kepada fungsi lain sebagai "callback" dan fungsi tersebut akan dipanggil kembali ketika diperlukan, seringkali sebagai respons terhadap suatu peristiwa.

Dalam Flutter, fungsi callback sering digunakan dalam berbagai konteks, seperti menangani interaksi pengguna, pembaruan data, atau penyelesaian operasi asinkron. Contoh umum penggunaan fungsi callback adalah dalam menangani aksi pengguna seperti ketika tombol ditekan atau ketika data diterima dari sumber eksternal seperti server.

Penggunaan fungsi callback memungkinkan Anda untuk membuat kode yang lebih terstruktur dan modular. Misalnya, Anda dapat memisahkan logika bisnis dari tampilan dengan menetapkan fungsi callback untuk menangani tindakan pengguna, sehingga memisahkan antara bagian UI dengan logika aplikasi.

Fungsi callback juga memungkinkan untuk membuat komponen yang dapat digunakan kembali dengan lebih baik. Anda dapat membuat widget yang menerima fungsi callback sebagai argumen, sehingga memungkinkan pengguna widget untuk menentukan tindakan yang akan dilakukan ketika suatu peristiwa terjadi.

Dengan menggunakan fungsi callback, Anda dapat membuat aplikasi Flutter yang lebih responsif dan dinamis. Misalnya, Anda dapat memberikan umpan balik langsung kepada pengguna saat mereka berinteraksi dengan aplikasi Anda, seperti menampilkan pesan pemberitahuan atau memicu animasi ketika aksi tertentu dilakukan.

Salah satu contoh penggunaan fungsi callback adalah dalam widget **GestureDetector**, di mana Anda dapat menetapkan fungsi callback untuk menanggapi berbagai gerakan pengguna seperti tap, drag, atau swipe. Ketika pengguna melakukan gerakan tertentu, fungsi callback yang sesuai akan dipanggil.

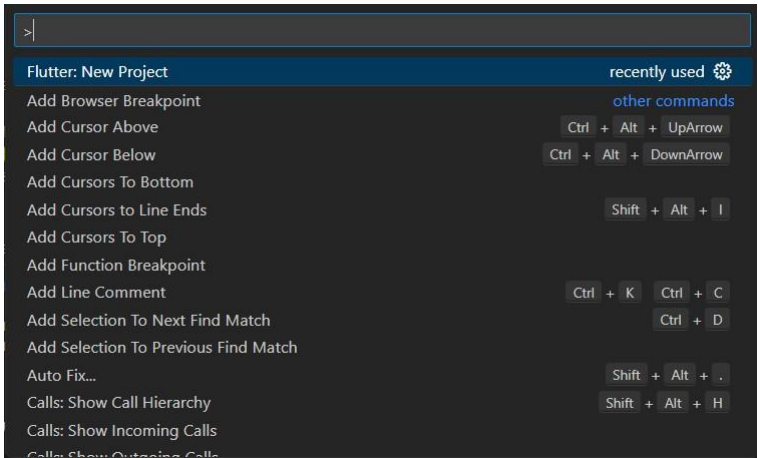
Penting untuk memperhatikan bahwa dalam Flutter, fungsi callback sering disampaikan sebagai argumen ke dalam widget atau metode lain, yang kemudian akan dipanggil oleh framework saat diperlukan. Oleh karena itu, pemahaman tentang bagaimana fungsi callback bekerja dalam konteks Flutter sangat penting untuk mengembangkan aplikasi yang efisien dan responsif.

Dalam pengembangan aplikasi Flutter yang lebih besar, penggunaan fungsi callback dapat membantu dalam memisahkan tanggung jawab dan menyederhanakan struktur kode. Ini memungkinkan pengembang untuk fokus pada satu aspek dari aplikasi pada satu waktu, dan membuat kode lebih mudah dipelihara dan diperbarui di masa mendatang.

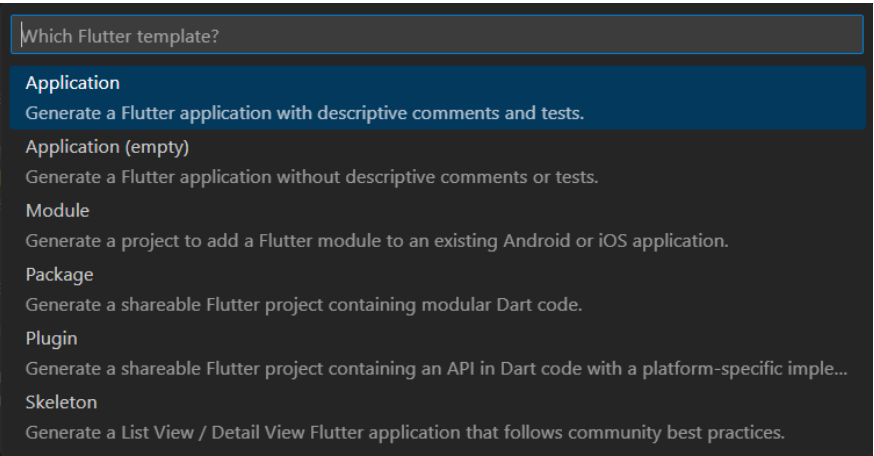
Dengan menggunakan fungsi callback secara bijaksana, Anda dapat meningkatkan fleksibilitas, modularitas, dan responsivitas aplikasi Flutter Anda. Ini memungkinkan Anda untuk membuat aplikasi yang lebih interaktif, dinamis, dan mudah diadaptasi sesuai dengan kebutuhan pengguna dan perubahan dalam aplikasi Anda.

Contoh coding:

Buat project baru menggunakan **Command Palette**. Pilih menu **View** dan klik **Command Palette** sehingga muncul tampilan berikut.

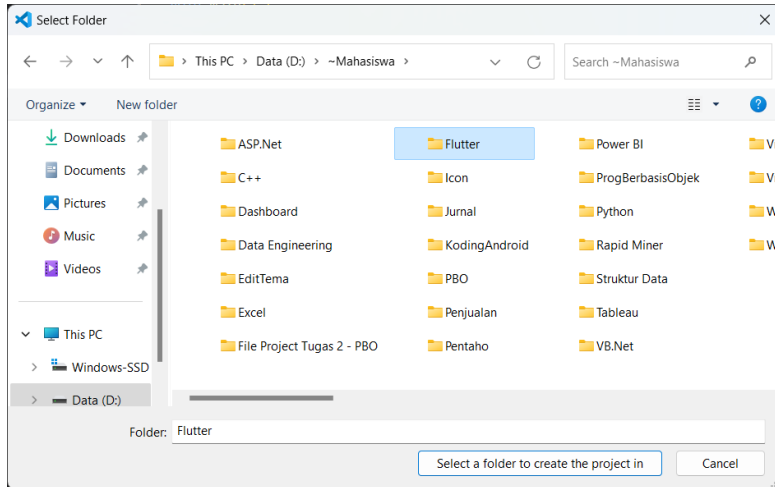


Kemudian ketikkan flutter, pilih **Flutter New Project**, maka akan muncul tampilan berikut.



Pilih menu **Application**.

Pilih folder yang diinginkan, kemudian klik tombol **Select a folder to create the project in**.



Beri nama projectnya, dengan mengetik **flutter_callback** kemudian tekan **Enter**.

Buka file **lib/main.dart**, hapus semua code yang ada karena kita akan memulainya dari awal. Kemudian masukkan code berikut:

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      title: 'Callback Example',
```

```

        home: CallbackExample(),
    );
}
}

```

```

class CallbackExample extends StatelessWidget {
  const CallbackExample({super.key});

  // Membuat fungsi callback yang menerima string sebagai argumen
  void _showMessage(String message) {
    debugPrint('Message received: $message');
    // Di sini, Anda dapat menambahkan logika lain sesuai kebutuhan
    aplikasi Anda
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Callback Example'),
    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          ElevatedButton(
            onPressed: () {
              // Memanggil fungsi callback saat tombol ditekan
              _showMessage('Button pressed');
            },
            child: const Text('Press Me'),
          ),
        ],
      ),
    ),
  );
}
}

```

Dalam contoh di atas, kita memiliki aplikasi Flutter sederhana dengan satu tombol yang akan memanggil sebuah fungsi callback saat tombol ditekan.

Fungsi callback kita bernama `_showMessage`, yang menerima sebuah string sebagai argumen. Di dalam fungsi callback ini, kita hanya mencetak pesan ke konsol untuk tujuan demonstrasi, tetapi Anda dapat menambahkan logika lain sesuai dengan kebutuhan aplikasi Anda, seperti menampilkan pesan pemberitahuan atau memicu perubahan tampilan.

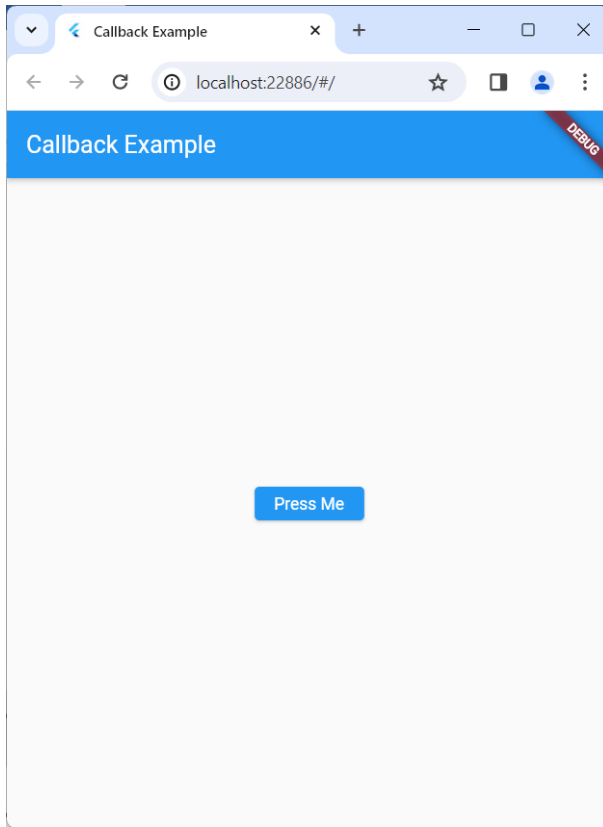
Ketika tombol ditekan, kita memanggil fungsi callback `_showMessage` dengan melewati string `'Button pressed'`. Ini memicu pemanggilan fungsi callback, yang kemudian akan mengeksekusi kode di dalamnya, seperti mencetak pesan ke konsol.

Dengan menggunakan fungsi callback seperti ini, Anda dapat mengintegrasikan logika pemrosesan ke dalam aplikasi Anda dengan cara yang modular dan fleksibel. Hal ini memungkinkan Anda untuk memisahkan tanggung jawab dan membuat kode Anda lebih mudah dipelihara dan diperluas di masa mendatang.

Untuk menjalankan koding, menggunakan terminal. Klik **New Terminal**. Kemudian ketikkan `flutter run`, tekan **Enter**. Pilih salah emulator yang akan digunakan untuk menjalankan project. Disini penulis menggunakan Chrome, berarti ketik angka 2.

```
PS D:\Mahasiswa\Flutter\aplikasipertama> flutter run
Multiple devices found:
Windows (desktop) • windows • windows-x64 • Microsoft Windows [Version 10.0.22621.1413]
Chrome (web)      • chrome • web-javascript • Google Chrome 111.0.5563.147
Edge (web)        • edge   • web-javascript • Microsoft Edge 111.0.1661.62
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (To quit, press "q/Q"): 2
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...
```

Jika tidak ada masalah, maka hasilnya akan tampil seperti berikut.



F. Menambahkan Animasi

Dalam pengembangan aplikasi Flutter, animasi dapat diterapkan pada berbagai elemen antarmuka pengguna, mulai dari perubahan posisi, ukuran, rotasi, hingga perubahan warna dan opasitas.

Salah satu keuntungan utama menggunakan animasi dalam aplikasi Flutter adalah bahwa Flutter menyediakan berbagai widget dan library yang memudahkan penggunaan dan implementasi animasi. Ini memungkinkan pengembang untuk menciptakan animasi yang kompleks dengan relatif mudah, bahkan tanpa pengetahuan khusus tentang animasi.

Dalam Flutter, animasi dapat diterapkan dengan menggunakan widget seperti **AnimatedContainer**, **AnimatedOpacity**, atau **AnimatedBuilder**.

Widget-widget ini memungkinkan Anda untuk menganimasikan perubahan properti tertentu dari widget, seperti ukuran, posisi, atau opasitas, dengan mulus.

Selain menggunakan widget bawaan Flutter, Anda juga dapat membuat animasi kustom menggunakan **AnimationController** dan beberapa jenis animasi yang disediakan oleh Flutter, seperti **Tween**, **CurvedAnimation**, dan banyak lagi. Dengan menggunakan **AnimationController**, Anda dapat mengontrol durasi, interpolasi, dan berbagai aspek lain dari animasi Anda.

Penggunaan animasi dalam aplikasi Flutter memerlukan pemahaman tentang konsep-konsep dasar animasi, seperti durasi, interpolasi, dan kurva perpindahan. Durasi mengacu pada berapa lama animasi berlangsung, sedangkan interpolasi mengontrol bagaimana nilai-nilai antara perubahan properti dihitung. Kurva perpindahan mengontrol bagaimana animasi berubah dari satu nilai ke nilai lainnya.

Selain itu, dalam menggunakan animasi dalam Flutter, penting untuk mempertimbangkan kinerja aplikasi Anda. Animasi yang terlalu rumit atau terlalu banyak dapat mempengaruhi kinerja aplikasi, terutama pada perangkat dengan spesifikasi rendah. Oleh karena itu, Anda harus menggunakan animasi dengan bijaksana dan mempertimbangkan kinerja saat merancang aplikasi Anda.

Animasi tidak hanya berguna untuk menarik perhatian pengguna, tetapi juga dapat membantu menyampaikan informasi dengan lebih baik. Misalnya, Anda dapat menggunakan animasi untuk memberikan umpan balik visual saat pengguna berinteraksi dengan elemen antarmuka pengguna, atau untuk menyampaikan transisi antara layar dalam aplikasi Anda.

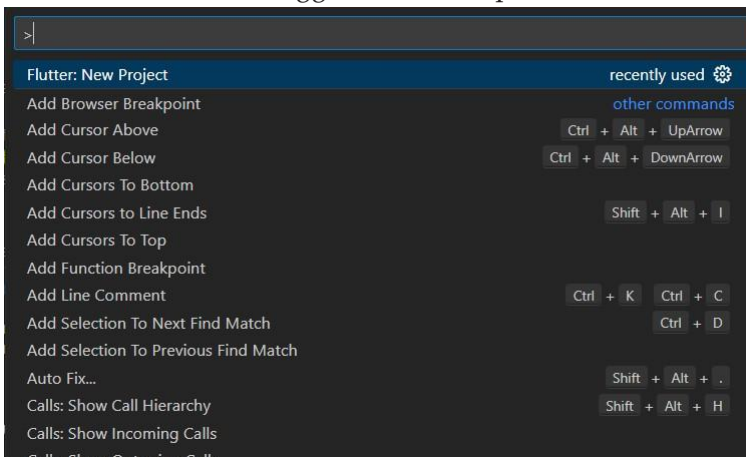
Penting juga untuk diingat bahwa animasi harus digunakan dengan bijaksana dan disesuaikan dengan konteks aplikasi Anda. Animasi yang berlebihan atau berlebihan dapat mengganggu pengalaman pengguna dan mengurangi kesan keseluruhan dari aplikasi Anda.

Seiring dengan menambahkan animasi, Anda juga harus memperhatikan antarmuka pengguna Anda secara keseluruhan. Animasi harus terintegrasi secara mulus dengan desain dan alur kerja aplikasi Anda, sehingga memberikan pengalaman pengguna yang konsisten dan intuitif.

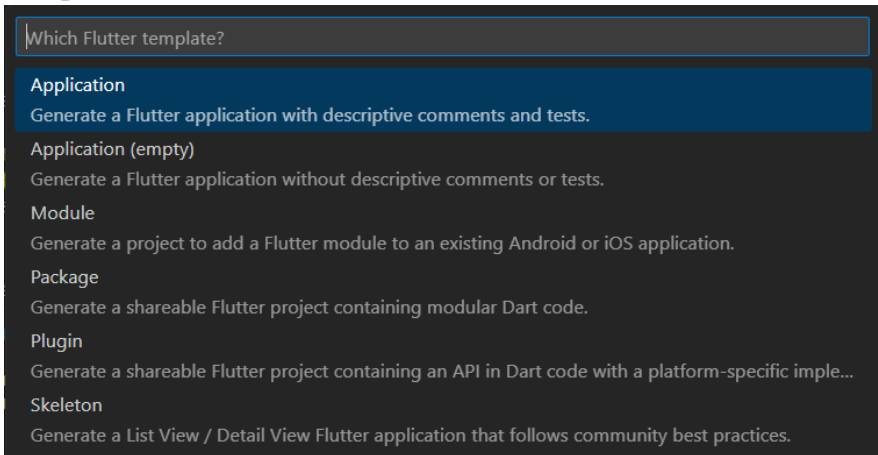
Pada akhirnya, menambahkan animasi ke dalam aplikasi Flutter adalah tentang meningkatkan interaksi dan pengalaman pengguna, serta membuat aplikasi Anda lebih menarik dan memuaskan untuk digunakan. Dengan menggunakan animasi dengan bijaksana dan memperhatikan prinsip desain yang baik, Anda dapat menciptakan aplikasi yang lebih menarik dan berkesan bagi pengguna Anda.

Contoh koding:

Buat project baru menggunakan **Command Palette**. Pilih menu **View** dan klik **Command Palette** sehingga muncul tampilan berikut.

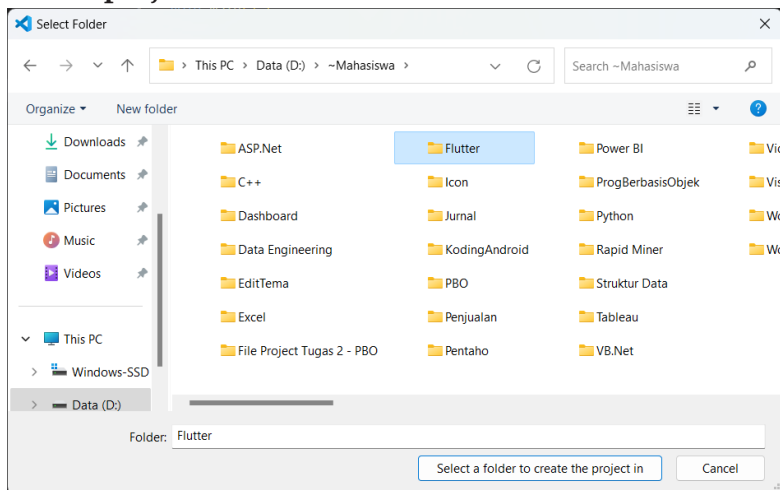


Kemudian ketikkan flutter, pilih **Flutter New Project**, maka akan muncul tampilan berikut.



Pilih menu **Application**.

Pilih folder yang diinginkan, kemudian klik tombol **Select a folder to create the project in**.



Beri nama projectnya, dengan mengetik **flutter_animasi** kemudian tekan **Enter**.

Buka file **lib/main.dart**, hapus semua code yang ada karena kita akan memulainya dari awal. Kemudian masukkan code berikut:

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      title: 'Animation Example',  
      home: AnimationExample(),  
    );  
  }  
}
```

```
class AnimationExample extends StatefulWidget {  
  const AnimationExample({super.key});
```

```
  @override  
  // ignore: library_private_types_in_public_api  
  _AnimationExampleState createState() => _AnimationExampleState();  
}
```

```
class _AnimationExampleState extends State<AnimationExample> {  
  Color _backgroundColor = Colors.blue;
```

```
  void _changeColor() {  
    setState(() {  
      // Ubah warna latar belakang secara acak  
      _backgroundColor = Colors.pink;  
    });  
  }
```



```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Animation Example'),
    ),
    body: Center(
      child: AnimatedContainer(
        duration: const Duration(seconds: 1),
        curve: Curves.easeInOut,
        color: _backgroundColor,
        width: 200,
        height: 200,
        child: const Center(
          child: Text(
            'Tap me!',
            style: TextStyle(
              fontSize: 20,
              fontWeight: FontWeight.bold,
              color: Colors.white,
            ),
          ),
        ),
      ),
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: _changeColor,
      child: const Icon(Icons.color_lens),
    ),
  );
}
```

Dalam contoh di atas, kita memiliki sebuah aplikasi Flutter sederhana yang menampilkan sebuah `AnimatedContainer` di tengah layar. Ketika tombol ditekan, warna latar belakang `AnimatedContainer` akan berubah secara mulus dari warna biru menjadi warna merah muda, dengan durasi animasi 1 detik.

Perhatikan penggunaan properti `duration` dan `curve` pada **`AnimatedContainer`**. Properti **`duration`** menentukan berapa lama animasi akan berlangsung, sementara properti **`curve`** mengontrol interpolasi animasi, dalam hal ini menggunakan **`Curves.easeInOut`** untuk memberikan efek perpindahan yang halus.

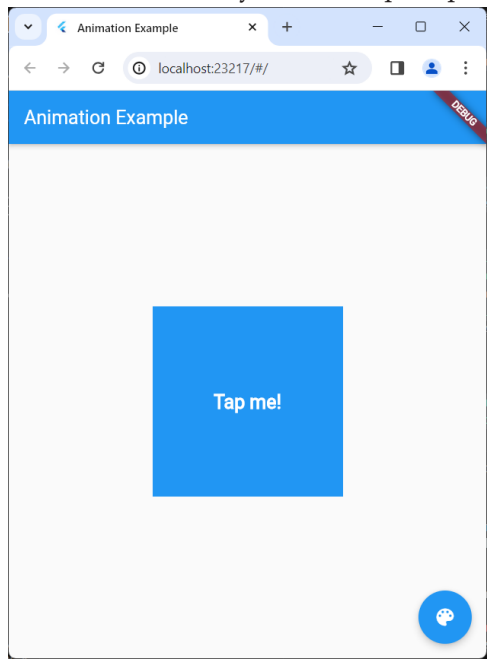
Ketika tombol floating action button ditekan, fungsi `_changeColor()` akan dipanggil, yang mengubah nilai warna latar belakang dan memicu pembaruan tampilan, sehingga memicu animasi perubahan warna.

Dengan menggunakan **`AnimatedContainer`** dan properti-propertinya, Anda dapat dengan mudah menambahkan animasi ke dalam aplikasi Anda, meningkatkan interaksi pengguna dan memberikan pengalaman yang lebih menarik dan dinamis.

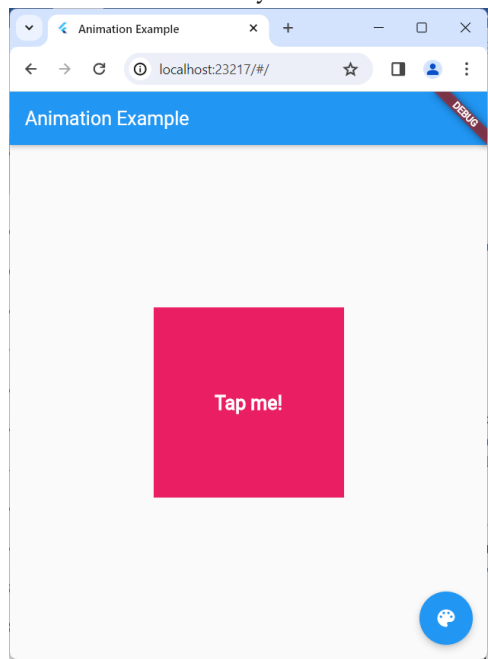
Untuk menjalankan koding, menggunakan terminal. Klik **New Terminal**. Kemudian ketikkan **`flutter run`**, tekan **Enter**. Pilih salah emulator yang akan digunakan untuk menjalankan project. Disini penulis menggunakan Chrome, berarti ketik angka 2.

```
PS D:\Mahasiswa\Flutter\aplikasipertama> flutter run
Multiple devices found:
Windows (desktop) • windows • windows-x64      • Microsoft Windows [Version 10.0.22621.1413]
Chrome (web)      • chrome • web-javascript • Google Chrome 111.0.5563.147
Edge (web)        • edge   • web-javascript • Microsoft Edge 111.0.1661.62
[1]: windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (To quit, press "q/Q"): 2
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...
```

Jika tidak ada masalah, maka hasilnya akan tampil seperti berikut.

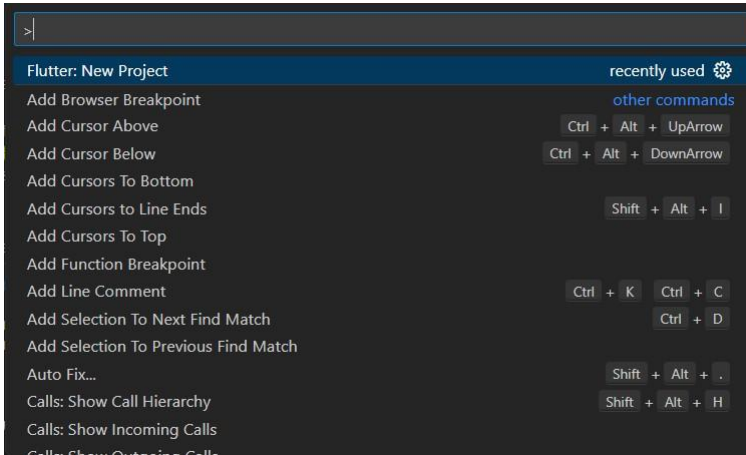


Ketika tombol ditekan, maka warnanya akan berubah.

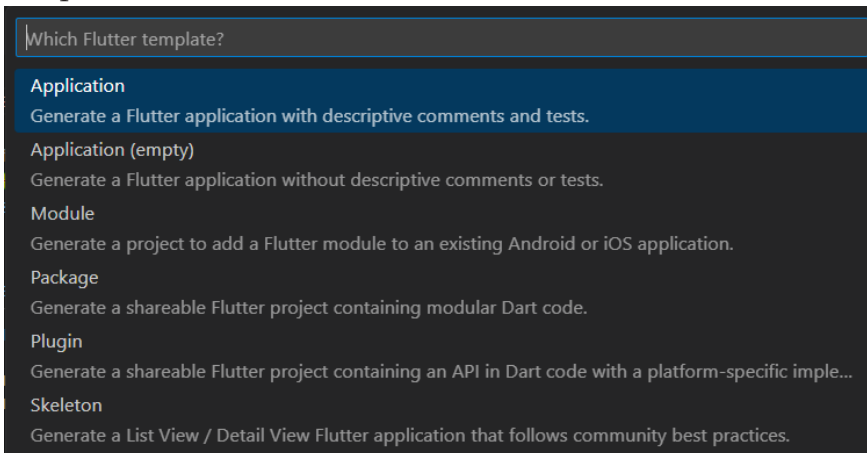


G. Contoh Koding: Membuat Tampilan Dinamis Untuk Aplikasi Flutter

Buat project baru menggunakan **Command Palette**. Pilih menu **View** dan klik **Command Palette** sehingga muncul tampilan berikut.

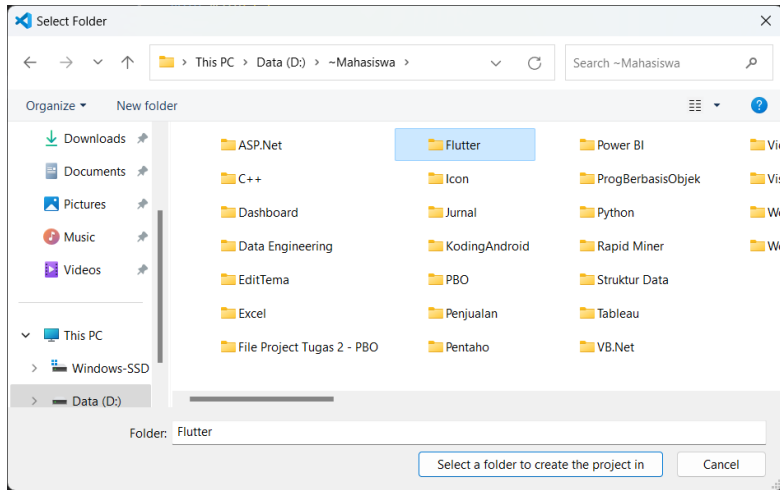


Kemudian ketikkan flutter, pilih **Flutter New Project**, maka akan muncul tampilan berikut.



Pilih menu **Application**.

Pilih folder yang diinginkan, kemudian klik tombol **Select a folder to create the project in**.



Beri nama projectnya, dengan mengetik **flutter_dinamis** kemudian tekan **Enter**.

Buka file **lib/main.dart**, hapus semua code yang ada karena kita akan memulainya dari awal. Kemudian masukkan code berikut:

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      title: 'Dynamic List Example',  
      home: DynamicListExample(),  
    );  
  }  
}
```

```
}
```

```
class DynamicListExample extends StatefulWidget {  
  const DynamicListExample({super.key});
```

```
  @override
```

```
    DynamicListExampleState createState() =>  
    DynamicListExampleState();  
}
```

```
class DynamicListExampleState extends State<DynamicListExample> {  
  List<String> items = ['Apple', 'Banana', 'Orange'];  
  TextEditingController textEditingController = TextEditingController();
```

```
  @override
```

```
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: const Text('Dynamic List Example'),  
      ),  
      body: Column(  
        children: <Widget>[  
          Expanded(  
            child: ListView.builder(  
              itemCount: items.length,  
              itemBuilder: (context, index) {  
                return ListTile(  
                  title: Text(items[index]),  
                );  
              },  
            ),  
          ),  
          Padding(  
            padding: const EdgeInsets.all(8.0),  
            child: Row(  

```

```

children: <Widget>[
  Expanded(
    child: TextField(
      controller: textEditingController,
      decoration: const InputDecoration(
        labelText: 'Add New Item',
      ),
    ),
  ),
  IconButton(
    icon: const Icon(Icons.add),
    onPressed: () {
      setState(() {
        items.add(textEditingController.text);
        textEditingController.clear();
      });
    },
  ),
],
),
],
);
}
}

```

Dalam contoh di atas, kita membuat aplikasi yang menampilkan daftar item dinamis. Pengguna dapat menambahkan item baru ke dalam daftar dengan menggunakan **TextField** dan tombol tambah.

Kelas **DynamicListExampleState** memiliki sebuah list items yang berisi daftar item dan sebuah **TextEditingController** untuk mengontrol input dari **TextField**.

Di dalam metode **build()**, kita menggunakan **ListView.builder** untuk menampilkan daftar item dalam bentuk list. Setiap item dirender sebagai **ListTile**.

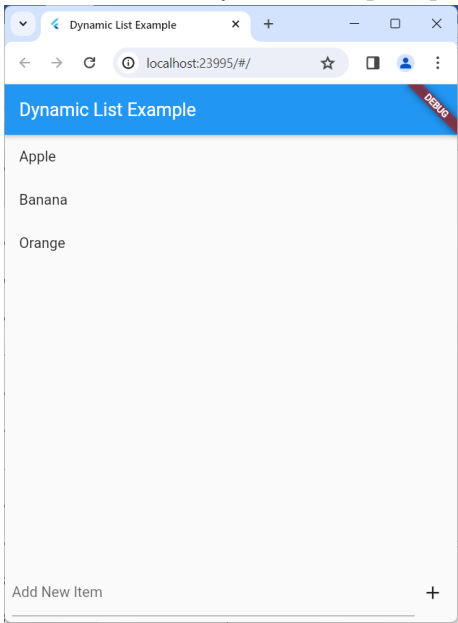
Pada bagian bawah tampilan, kita menempatkan **TextField** yang memungkinkan pengguna untuk memasukkan item baru. Ketika tombol tambah ditekan, kita menambahkan item baru ke dalam list menggunakan **setState()** untuk memperbarui tampilan.

Dengan demikian, pengguna dapat dengan mudah menambahkan item baru ke dalam daftar secara dinamis. Semoga contoh ini membantu Anda memahami cara membuat tampilan dinamis dengan Flutter.

Untuk menjalankan koding, menggunakan terminal. Klik **New Terminal**. Kemudian ketikkan **flutter run**, tekan **Enter**. Pilih salah emulator yang akan digunakan untuk menjalankan project. Disini penulis menggunakan Chrome, berarti ketik angka 2.

```
PS D:\~Mahasiswa\Flutter\aplikasipertama> flutter run
Multiple devices found:
Windows (desktop) • windows • windows-x64 • Microsoft Windows [Version 10.0.22621.1413]
Chrome (web)      • chrome • web-javascript • Google Chrome 111.0.5563.147
Edge (web)        • edge   • web-javascript • Microsoft Edge 111.0.1661.62
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (To quit, press "q/Q"): 2
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...
```


Jika tidak ada masalah, maka hasilnya akan tampil seperti berikut.



Ketika kolom “Add New Item” diketikkan “Tambahan Buah” dan ditekan tombol “+”, maka hasilnya akan seperti di bawah ini.

